

lists

CSCI
134

after registration, sometimes
we need to "rebalance" lecture sections

Class	Section	Days & Times	Room	Instructor	Meeting Dates	Status
3267	01-LEC Regular	MoWeFr 9:00AM - 9:50AM	Schow Library Classroom 030A	Rohit Bhattacharya	02/02/2023 - 05/12/2023	38
Class	Section	Days & Times	Room	Instructor	Meeting Dates	Status
3268	02-LEC Regular	MoWeFr 10:00AM - 10:50AM	Schow Library Classroom 030A	Rohit Bhattacharya	02/02/2023 - 05/12/2023	54
Class	Section	Days & Times	Room	Instructor	Meeting Dates	Status
3276	03-LEC Regular	MoWeFr 11:00AM - 11:50AM	Schow Library Classroom 030A	Mark Hopkins	02/02/2023 - 05/12/2023	7

to do this with python, we could
use a way to represent all the
registered students

each student can be
represented as a string

but how do we
represent the entire
collection?

```
"Samskruthi Attada"  
"Sim Bethel"  
"Sarah Evellyn Borges"  
"Chase Bradshaw"  
"Brian Coleman"  
"Emery Dunbar"  
"Devansh Dutta"  
"Silas Fleissig"  
"Tong Gao"  
"Andrew Gu"  
"Kai Jackson"  
"Liam Jones"
```

as a list

```
["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```


list

```
[ expression , expression , expression , expression , ... , expression ]
```

a list can contain
zero or more
expressions

```
In [1]: ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
Out[1]: ['Attada', 'Bethel', 'Borges', 'Bradshaw', 'Coleman', 'Dunbar']

In [2]: [100, 200, 300]
Out[2]: [100, 200, 300]

In [3]: [3.14, 100, "hello", True, lambda x: x * 2]
Out[3]: [3.14, 100, 'hello', True, <function __main__.<lambda>(x)>]

In [4]: []
Out[4]: []
```

just as we can compute the length of a str using the len function, we can also compute the length of a list

```
In [1]: len(["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"])  
Out[1]: 6
```

```
In [2]: len([100, 200, 300])  
Out[2]: 3
```

```
In [3]: len([3.14, 100, "hello", True, lambda x: x * 2])  
Out[3]: 5
```

```
In [4]: len([])  
Out[4]: 0
```

we can refer to individual elements of a list using their **position** (a.k.a. **index**)

```
students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```

↑ ↑ ↑ ↑ ↑ ↑
0 **1** **2** **3** **4** **5**

but we start counting from **zero**

the value of

`list` [`int`]

is the element at
position `int`
of the `list`

```
In [2]: students[0]
Out[2]: 'Attada'

In [3]: students[1]
Out[3]: 'Bethel'

In [4]: students[2]
Out[4]: 'Borges'

In [5]: students[3]
Out[5]: 'Bradshaw'

In [6]: students[4]
Out[6]: 'Coleman'

In [7]: students[5]
Out[7]: 'Dunbar'
```

```
students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```

↑
`0`

↑
`1`

↑
`2`

↑
`3`

↑
`4`

↑
`5`

how to we obtain the sections?

we
want



```
section1 = ['Attada', 'Bethel']  
section2 = ['Borges', 'Bradshaw']  
section3 = ['Coleman', 'Dunbar']
```

```
students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```

↑
0

↑
1

↑
2

↑
3

↑
4

↑
5

the value of `list [ int1 : int2 ]` is the list of elements from position `int1` up to (but not including) position `int2` in `list`

```
In [1]: students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```

```
In [2]: section1 = students[0:2]
```

```
In [3]: section2 = students[2:4]
```

```
In [4]: section3 = students[4:6]
```

```
In [5]: section1
```

```
Out[5]: ['Attada', 'Bethel']
```

```
In [6]: section2
```

```
Out[6]: ['Borges', 'Bradshaw']
```

```
In [7]: section3
```

```
Out[7]: ['Coleman', 'Dunbar']
```

but perhaps we want to diversify the sections
(e.g. not group them alphabetically)

```
section1 = ['Attada', 'Bethel']  
section2 = ['Borges', 'Bradshaw']  
section3 = ['Coleman', 'Dunbar']
```



```
section1 = ['Attada', 'Bradshaw']  
section2 = ['Bethel', 'Coleman']  
section3 = ['Borges', 'Dunbar']
```

```
students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```

↑
0

↑
1

↑
2

↑
3

↑
4

↑
5

the value of `list [ int1 : int2 : int3 ]` is the list of elements from position `int1` up to (but not including) position `int2` in `list` but only including every `int3` th element

```
In [1]: students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
In [2]: section1 = students[0:6:3]
In [3]: section2 = students[1:6:3]
In [4]: section3 = students[2:6:3]

In [5]: section1
Out[5]: ['Attada', 'Bradshaw']

In [6]: section2
Out[6]: ['Bethel', 'Coleman']

In [7]: section3
Out[7]: ['Borges', 'Dunbar']
```


list indexing

```
list [ int ]
```

the element at position

```
int in list
```

list slicing

```
list [ int1 : int2 ]
```

elements from position `int1` up to (but not including) position `int2` in `list`

```
list [ int1 : int2 : int3 ]
```

elements from position `int1` up to (but not including) position `int2` in `list`
but only including every `int3` th element

special cases

list indexing

`list` [`int`]

the element at position

`int` in `list`

if **negative**, this refers to the element
at position $\text{len}(\text{list}) + \text{int}$

```
In [1]: students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
In [2]: students[-1]
Out[2]: 'Dunbar'

In [3]: students[-2]
Out[3]: 'Coleman'

In [4]: students[-3]
Out[4]: 'Bradshaw'
```

list slicing

`list [ int1 : int2 ]`

elements from position `int1` up to (but not including) position `int2` in `list`

special cases

if omitted, python assumes
"start at position zero"

if omitted, python assumes
"include everything up to (and including) the final element"

```
In [1]: students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
In [2]: students[0:4]
Out[2]: ['Attada', 'Bethel', 'Borges', 'Bradshaw']
In [3]: students[:4]
Out[3]: ['Attada', 'Bethel', 'Borges', 'Bradshaw']
In [4]: students[3:6]
Out[4]: ['Bradshaw', 'Coleman', 'Dunbar']
In [5]: students[3:]
Out[5]: ['Bradshaw', 'Coleman', 'Dunbar']
In [6]: students[:]
Out[6]: ['Attada', 'Bethel', 'Borges', 'Bradshaw', 'Coleman', 'Dunbar']
```

special cases

list slicing

`list [ int1 : int2 : int3 ]`



elements from position `int1` up to (but not including) position `int2` in `list` but only including every `int3` th element

if **negative**, the expression provides the elements from position `int1` until (but not including) position `int2` in `list`, skipping back by `|int3|` elements each time

```
In [1]: students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
```

```
In [2]: students[5:2:-1]
```

```
Out[2]: ['Dunbar', 'Coleman', 'Bradshaw']
```

```
In [3]: students[4:0:-2]
```

```
Out[3]: ['Coleman', 'Borges']
```

```
In [4]: students[4:0:-1]
```

```
Out[4]: ['Coleman', 'Bradshaw', 'Borges', 'Bethel']
```

special cases

when negative

list slicing

`list [ int1 : int2 : int3 ]`

elements from position `int1` up to (but not including) position `int2` in `list` but only including every `int3` th element

if omitted, python assumes "start at the final position"

if omitted, python assumes "include everything up to (and including) element zero"

```
In [1]: students = ["Attada", "Bethel", "Borges", "Bradshaw", "Coleman", "Dunbar"]
In [2]: students[4::-1]
Out[2]: ['Coleman', 'Bradshaw', 'Borges', 'Bethel', 'Attada']

In [3]: students[:2:-1]
Out[3]: ['Dunbar', 'Coleman', 'Bradshaw']

In [4]: students[::-1]
Out[4]: ['Dunbar', 'Coleman', 'Bradshaw', 'Borges', 'Bethel', 'Attada']
```

concatenation

```
In [1]: ["Attada", "Bethel", "Borges"] + ["Bradshaw", "Coleman"]  
Out[1]: ['Attada', 'Bethel', 'Borges', 'Bradshaw', 'Coleman']  
  
In [2]: ["Attada", "Bethel", "Borges"] + ["Bradshaw"]  
Out[2]: ['Attada', 'Bethel', 'Borges', 'Bradshaw']
```

like strs, lists can be **added** together

concatenation

```
In [1]: ["Attada", "Bethel", "Borges"] + ["Bradshaw", "Coleman"]  
Out[1]: ['Attada', 'Bethel', 'Borges', 'Bradshaw', 'Coleman']
```

```
In [2]: ["Attada", "Bethel", "Borges"] + ["Bradshaw"]  
Out[2]: ['Attada', 'Bethel', 'Borges', 'Bradshaw']
```

```
In [3]: ["Attada", "Bethel", "Borges"] + "Bradshaw"
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[3], line 1  
----> 1 ["Attada", "Bethel", "Borges"] + "Bradshaw"  
  
TypeError: can only concatenate list (not "str") to list
```



like strs, lists can be **added** together
but they cannot be added with other expression types

we can **iterate** over the elements of a list

```
for variable in range(int, int):
```

python code

```
for variable in list:
```

python code

```
In [1]: names = ["cm", "otg", "bh"]
```

```
In [2]: for i in range(len(names)):
...:     print(names[i])
...:
```

```
cm
otg
bh
```

iPython

```
In [1]: names = ["cm", "otg", "bh"]
```

```
In [2]: for name in names:
...:     print(name)
...:
```

```
cm
otg
bh
```

iPython

expressions so far

type	example	value
int	7	7
float	3.14	3.14
str	"williams"	"williams"
bool	3 > 4	False
function	abs	<code><function abs(x)></code>
NoneType	print("hi")	None
range	range(1,5)	range(1,5)
list	["mh", "rb", "jb"]	["mh", "rb", "jb"]